parking lot tracker part b submission

name: ezlan sohani

date: 2026-10-11

github: https://github.com/ssscaryjoe/ezparking

contents:

1. brief pages with part a and b actual tc01-tc08, reflection and checklist
2. test shots 1-4 appended
3. source files in repo and zip

SUZANNE CORY
High School

Data structures and Object-Oriented Programming Task
Level 2 IT (2026)

Parking Lot Tracker — Two-Part Python Programming Assessment

Name:	Ezlan Sohani	**Home Group:**	
Teacher:		**Due date:**	

ASSESSMENT OVERVIEW

You will design, implement, test and evaluate the same Parking Lot Tracker twice. Part A uses structured data and functions. Part B redesigns the solution using encapsulated classes and objects. The aim is to demonstrate that you can solve the same problem using two different software designs.

Submission requirements

- parking_lot_tracker_part_a.py — completed procedural program
- parking_lot_tracker_part_b.py — completed object-oriented program
- parking_data.txt — data file used by both solutions (or two clearly named equivalent files)
- Completed IPO chart and pseudocode for major functions/methods
- Screen captures showing successful program testing
- Short comparison explaining how the data and behaviour changed between Part A and Part B

github: https://github.com/ssscaryjoe/ezparking

1. Project scope, requirements and constraints

Scope

The Parking Lot Tracker records whether parking bays in a small car park are occupied or free. It must allow a car to enter a bay, remove a car, display the status of every bay, calculate summary statistics, load saved data at startup and save changes before exiting.

The system does not include payment processing, sensors, networking, external databases, graphical interfaces or real-time updates. These features are outside the scope of this assessment.

Functional requirements

- Allow the user to record a car entering a valid, currently free parking bay.
- Allow the user to remove a car from an occupied bay.
- Display every bay number, occupancy status and licence plate where appropriate.
- Calculate the total number of bays, occupied bays and free bays.
- Prevent inconsistent actions, such as parking in an occupied bay or removing a car from an empty bay.

Non-functional requirements

- Both programs must run in a standard Python interpreter.
- Output must be clear, readable and appropriate for a text-based interface.
- Code must be modular, consistently named, internally documented and maintainable.
- Part B must demonstrate encapsulation rather than simply placing the Part A dictionaries inside a class.

Required menu

Option	Action
1	Record a car entering a bay
2	Remove a car from a bay
3	View status of all bays
4	Display occupancy totals
5	Save data and exit

2. Part A — Procedural solution 50%

Purpose

Create a complete function-based Python program. The program must use a dictionary of dictionaries as its main data structure. The outer dictionary identifies each parking bay; each inner dictionary stores the data belonging to that bay.

Required main data structure

```
dictBays = {
    1: {"plate": "", "occupied": False},
    2: {"plate": "", "occupied": False},
    3: {"plate": "", "occupied": False}
}
```

You may extend the inner dictionaries with additional relevant fields. The bay number may be stored as the outer key rather than repeated inside every inner dictionary.

Alternative structure

A list of lists is not the intended solution. Students who believe it is necessary must discuss their design with the teacher before coding. Even with approval, this structure limits the Part A data-structure/design criterion to a maximum of Band 2.

Part A program requirements

- Use functions to separate major tasks.
- Use at least one while loop for the menu and at least one for loop for processing bays.
- Use dictionary access and updates correctly.
- Use counters and/or summing logic to calculate occupancy statistics.
- Use a flag or another clear method where a search or validation process requires one.
- Use the required naming and documentation conventions.

Minimum functions

Function	Purpose	Minimum responsibility
displayMenu()	Display choices	Show the five menu options.
loadData()	Load saved state	Read parking_data.txt and build/update dictBays.
saveData()	Save current state	Write the current bay data to parking_data.txt.
recordEntry()	Park a car	Validate bay and plate; reject occupied bays; update the inner dictionary.
removeCar()	Remove a car	Validate bay; reject empty bays; clear plate and occupied state.
viewStatus()	Display bays	Show all bays in a readable format.
calculateTotals()	Calculate statistics	Display total, occupied and free bay counts.

Part A evidence to submit

- IPO chart
- Pseudocode for each major function
- Completed Python program
- Evidence of debugging and at least one corrected error

3. Part B — Object-oriented solution with encapsulation 50%

Purpose

Redesign the Parking Lot Tracker using classes and objects. You may use a list of objects as the main collection. Each parking bay must be represented by an object whose data is protected through encapsulation.

Required class design

```
class ParkingBay:
    def __init__(self, bayNumber):
        self.__bayNumber = bayNumber
        self.__plate = ""
        self.__occupied = False

    # Add appropriate public methods here.
```

Encapsulation requirements

- Bay data must be stored in private attributes, such as __bayNumber, __plate and __occupied.
- Code outside the class must not directly change private attributes.
- Provide public methods that control valid changes to an object.
- Getter methods may be used when information needs to be displayed or saved.
- Setter methods should only be included where they protect validity; do not create unrestricted setters merely to expose every private attribute.
- The class should prevent invalid state changes where practical.

Expected object behaviours

Suggested method	Purpose	Encapsulation expectation
__init__(bayNumber)	Create a free bay	Initialise private attributes.
parkCar(plate)	Occupy a free bay	Reject an empty plate or an already occupied bay; change state only when valid.
removeCar()	Free an occupied bay	Reject an empty bay; clear the plate and occupancy state when valid.
isOccupied()	Report occupancy	Return the private Boolean value without exposing direct modification.
getBayNumber()	Report bay number	Return the bay identifier.
getPlate()	Report plate	Return the plate for display or saving.

Required collection

```
lstBays = [
    ParkingBay(1),
    ParkingBay(2),
    ParkingBay(3)
]
```

Part B program requirements

- Provide the same user features and validation as Part A.
- Use ParkingBay objects rather than bay dictionaries.
- Use a list of objects to store the parking bays.
- Use object methods to park and remove cars.
- Load file data by creating or updating objects through public methods.

- Save object data using getter methods or a suitable serialisation method.
- Maintain clear functions for menu control, file handling and collection-level operations.
- Demonstrate that the object-oriented version is not merely a renamed copy of the procedural solution.

Optional extension

Students may create a second class, such as ParkingLot, to manage the collection of ParkingBay objects. This is optional and should only be used when it improves cohesion and clarity.

4. Planning and design evidence

IPO chart

Input	Process	Output
Menu choice	Repeat menu and validate selection	Text menu / error message
Bay number	Locate matching dictionary entry or object	Selected bay / invalid-bay message
Licence plate	Validate and update bay state	Confirmation / rejection message
Text-file data	Parse data and populate program structure	Loaded parking status
Current bay collection	Count occupied and free bays	Summary statistics

Pseudocode requirements

- Write pseudocode before implementation for the main program and all major functions in Part A.
- For Part B, write pseudocode for the class methods and the main program operations that use the list of objects.
- Your pseudocode may change during development, but the final version must match the submitted program logic.

Comparison reflection

Write approximately 200–300 words answering the following:

- How was each parking bay represented in Part A and Part B?
- How did encapsulation change the way bay data was accessed and modified?
- Which version was easier to extend or maintain, and why?
- Identify one benefit and one limitation of each design.

In Part A, each parking bay was represented as an inner dictionary containing keys for plate and occupied status, stored within an outer dictionary keyed by bay number. In Part B, each bay was modeled as an instance of a ParkingBay class stored in a list (lstBays), holding private attributes (__bayNumber, __plate, and __occupied).

Encapsulation fundamentally changed how data was accessed and modified. In Part A, external functions directly modified dictionary values (e.g. dictBays[bay]["occupied"] = True). In Part B, code outside the class cannot access private attributes directly and must interact through public methods (parkCar(), removeCar(), isOccupied(), and getPlate()). This ensured that state changes were always validated internally before data was updated, preventing inconsistent states such as clearing an occupancy flag while leaving an old plate stored.

Part B is significantly easier to maintain and extend. If new requirements are introduced in the future, such as tracking vehicle types or arrival times, internal attributes and validation rules can be added directly inside the ParkingBay class without modifying external procedural code or breaking menu handling.

A major benefit of Part A was its simplicity, as dictionaries are fast to configure for small programs; however, its main limitation was lack of data protection, as any function could accidentally overwrite or corrupt bay records. Conversely, the main benefit of Part B is robust data protection and encapsulation, with the only drawback being the extra initial boilerplate of defining classes and getter methods.

5. Testing and evidence

Complete this testing plan. Tests may be reused where the expected behaviour is the same, but actual results must be recorded separately for Part A and Part B.

ID	Feature / input	Test type	Expected result	Part A actual	Part B actual
TC01	Park valid car in a free bay	Normal	Bay becomes occupied and plate is stored.	bay 2 occupied xyz789 pass	bay 2 occupied xyz789 pass
TC02	Remove car from occupied bay	Normal	Bay becomes free and plate is cleared.	bay 1 free pass	bay 1 free pass
TC03	First valid bay number	Boundary	First bay can be updated correctly.	bay 1 updated pass	bay 1 updated pass
TC04	Last valid bay number	Boundary	Last bay can be updated correctly.	bay 3 updated pass	bay 3 updated pass
TC05	Bay number outside range	Invalid	Error shown; no data changes.	invalid bay no change pass	invalid bay no change pass
TC06	Empty licence plate	Invalid	Entry rejected; no data changes.	invalid plate no change pass	invalid plate no change pass
TC07	Park in occupied bay	Inconsistent action	Entry rejected; existing plate remains.	rejected abc123 kept pass	rejected abc123 kept pass
TC08	Save, close and reload	File handling	Saved occupancy is restored correctly.	reloaded occupied pass	reloaded occupied pass

Additional testing requirements

- Include at least one test for a completely empty car park.
- Include at least one test for a full car park.
- Record at least one bug found, its cause and how it was corrected.
- Provide screen captures showing both versions running successfully.

6. Naming and internal documentation

Naming conventions

Element	Convention	Example
Functions and methods	camelCase	recordEntry(), calculateTotals(), parkCar()
Classes	PascalCase	ParkingBay, ParkingLot
Private attributes	double underscore + clear name	self.__plate, self.__occupied
Integer counters / indices	Hungarian notation permitted	i_count, i_index
Boolean flags	Hungarian notation permitted	b_found, b_running
Collections	clear Hungarian-style prefix permitted	dictBays, lstBays

Required header comment

```
# File: parking_lot_tracker_part_a.py
# Description: Tracks parking-bay occupancy using a dictionary of dictionaries.
# Author: <Student Name>
# Date created: <Date>
```

Part B should use an equivalent header that identifies the object-oriented implementation.

Function and method documentation

```
# Function / Method: parkCar()
# Purpose: Occupy a free parking bay with a validated licence plate.
# Inputs: plate (string)
# Outputs: success status and/or message
# Data types: string, bool
```

7. Assessment criteria and performance bands

The descriptors below are applied using the evidence submitted across planning, implementation, testing and evaluation. A student does not need to use identical code to the examples in this booklet, but the required design constraints must be met.

Criterion	Band 4	Band 3	Band	Band 4
Part A data structure and manipulation	Program structure is incomplete or data is not stored/manipulated reliably.	Uses a basic structure with partial success. An approved list-of-lists solution cannot exceed this band.	Correctly uses a dictionary of dictionaries for most required operations.	Uses a well-designed dictionary of dictionaries consistently, accurately and efficiently.
Part A procedural design	Limited decomposition; functions or control structures are incomplete.	Some suitable functions and loops, but responsibilities or data flow are inconsistent.	Functions, loops, validation and file handling are mostly clear and correct.	Highly cohesive functions, clear control flow, robust validation and maintainable procedural design.
Part B class and object design	Class is incomplete or objects are not used meaningfully.	Uses a class and objects, but behaviour remains mostly external or data is directly exposed.	Uses ParkingBay objects with appropriate attributes and methods; most behaviour is correctly placed.	Class responsibilities are cohesive; objects fully model bay state and behaviour with strong design decisions.
Encapsulation	Attributes are public or directly modified throughout the program.	Private attributes are present, but external code bypasses them or access methods provide weak control.	Private attributes and public methods generally protect valid state changes.	Encapsulation is consistently enforced; methods validate changes and expose only necessary information.
Correctness and validation	Major requirements do not work or invalid input frequently causes failure.	Core features work in simple cases; validation is limited or inconsistent.	Most functional requirements work and invalid/inconsistent actions are handled.	All core requirements work reliably with comprehensive, user-friendly validation.
File handling, testing and debugging	Little evidence of saving/loading or systematic testing.	Basic file handling and several tests are shown, with limited analysis.	File handling is reliable; test plan includes normal, boundary and invalid cases.	Thorough testing, clear actual results and strong evidence of diagnosing and correcting defects.
Documentation and evaluation	Documentation is missing or does not explain the solution.	Basic comments and partial planning/evaluation are present.	Clear naming, comments, pseudocode and a sound comparison of the two designs.	Precise documentation and insightful evaluation using appropriate software-development terminology.

Maximum-band rule for Part A

A list-of-lists solution, even when discussed with and approved by the teacher, cannot receive higher than Band 2 for “Part A data structure and manipulation”. Other criteria may still be assessed at their demonstrated level.

Band 5 will be assigned when the criterion is insufficient for Band 4

8. Student submission checklist

- ☐ Part A uses a dictionary of dictionaries as the main data structure.
- ☒ Part A includes all required menu options, functions, and validation.
- ☒ Part B uses a class with private attributes and a list of objects.
- ☒ Part B changes object state through methods rather than direct attribute access.
- ☒ IPO chart and pseudocode are complete and match the final programs.
- ☒ Test plans contain expected and actual results for normal, boundary and invalid cases.
- ☒ I have included evidence of at least one bug found and corrected.
- ☒ I have completed the comparison reflection.
- ☒ All files are clearly named and uploaded to Compass.

Teacher approval for alternative Part A structure (only if applicable)

Proposed structure	Reason for request
Teacher decision / conditions	

END OF ASSESSMENT BOOKLET

part b test shot 1 - empty lot totals and status

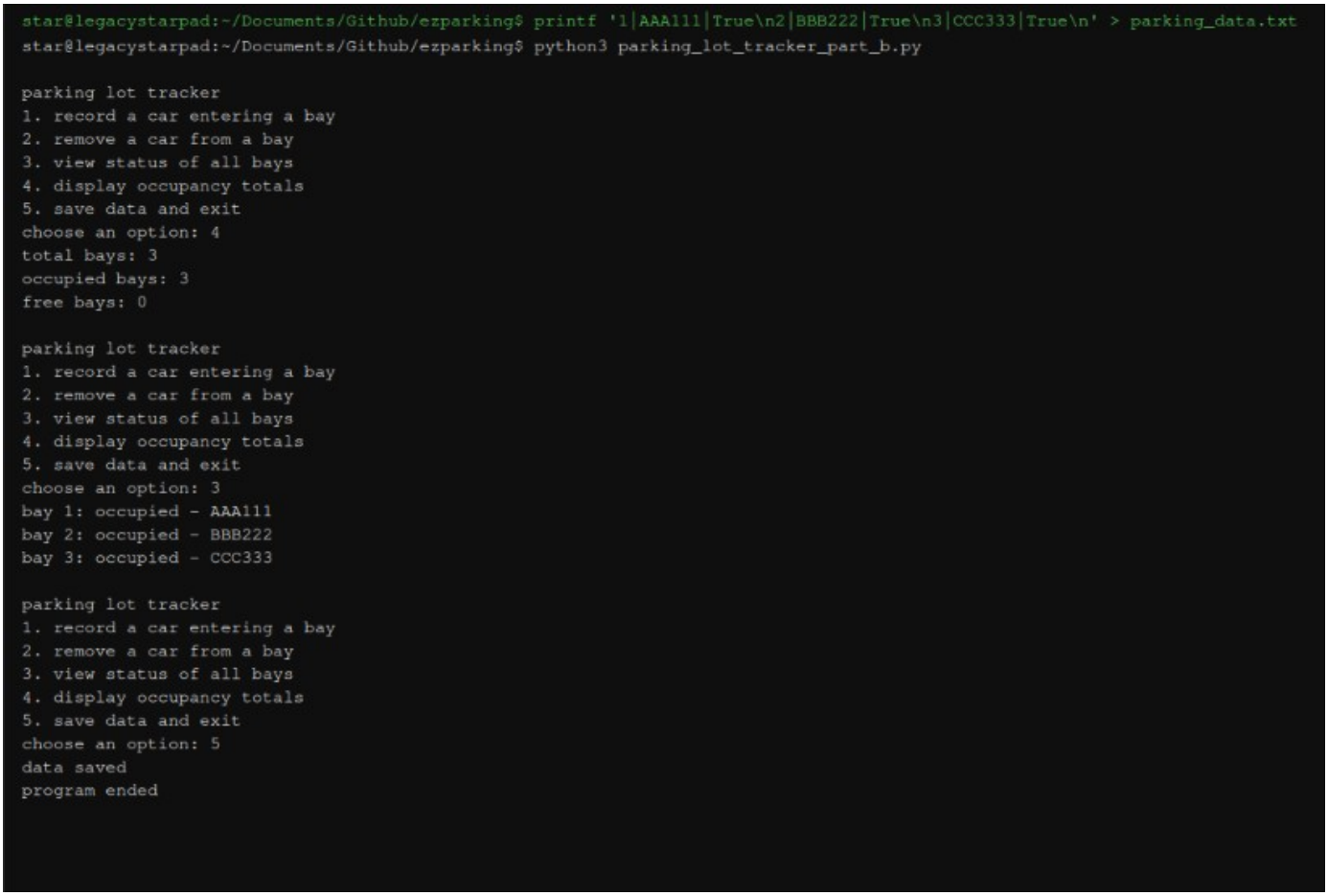
```
star@legacystarpad:~/Documents/Github/ezparking$ printf '1||False\n2||False\n3||False\n' > parking_data.txt
star@legacystarpad:~/Documents/Github/ezparking$ python3 parking_lot_tracker_part_b.py

parking lot tracker
1. record a car entering a bay
2. remove a car from a bay
3. view status of all bays
4. display occupancy totals
5. save data and exit
choose an option: 4
total bays: 3
occupied bays: 0
free bays: 3

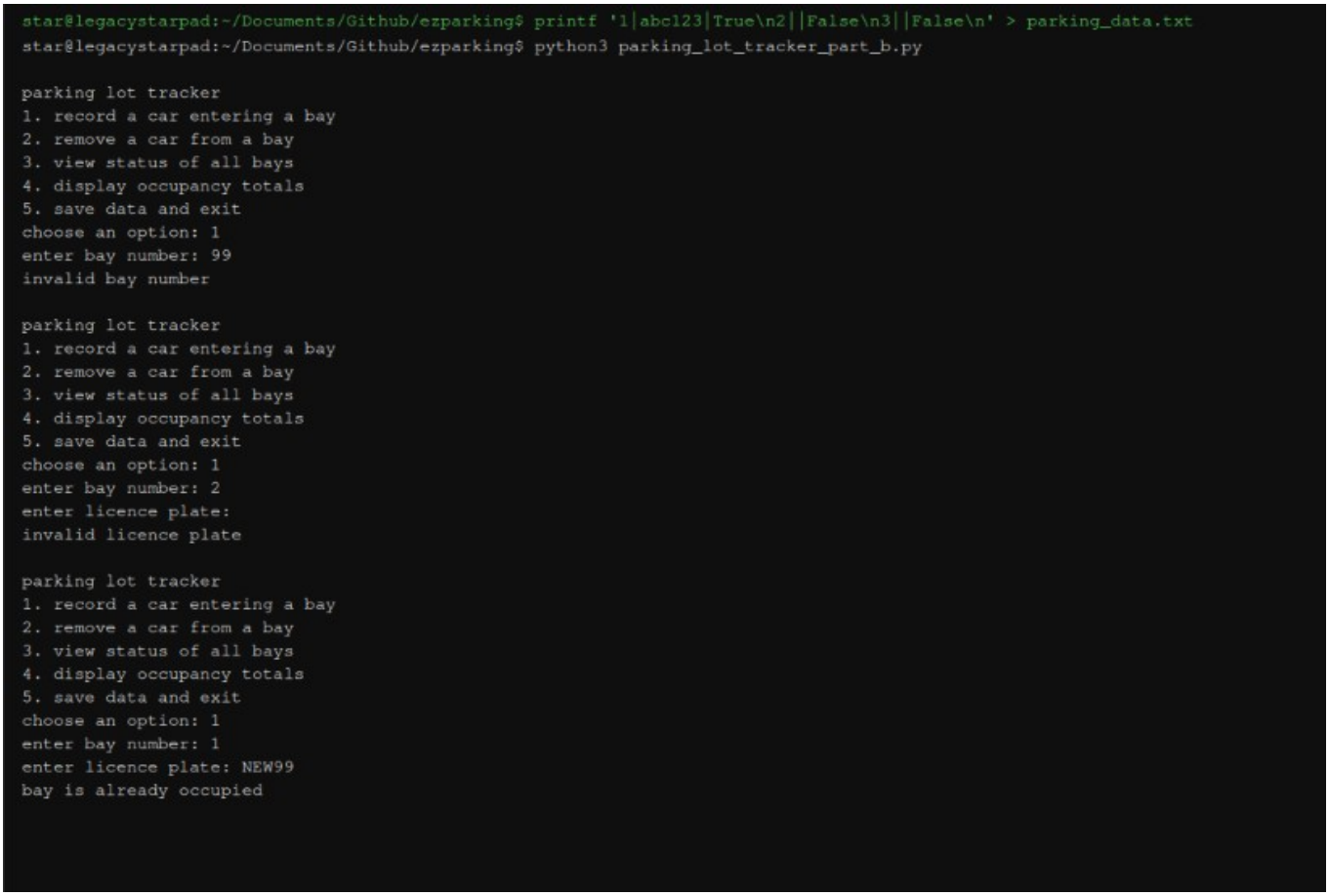
parking lot tracker
1. record a car entering a bay
2. remove a car from a bay
3. view status of all bays
4. display occupancy totals
5. save data and exit
choose an option: 3
bay 1: free
bay 2: free
bay 3: free

parking lot tracker
1. record a car entering a bay
2. remove a car from a bay
3. view status of all bays
4. display occupancy totals
5. save data and exit
choose an option: 5
data saved
program ended
```

part b test shot 2 - full lot totals and status



part b test shot 3 - invalid bay empty plate occupied rejection



part b test shot 4 - normal entry xyz789 saved and reloaded

